

Labo 02 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — Noms complets et noms courts

Réponse.

Écriture	Nom complet	Podman
<code>nginx</code>	<code>docker.io/library/nginx:latest</code>	alias connu → résolu sans question, message <code>Resolved "nginx" as an alias</code>
<code>bitnami/nginx</code>	<code>docker.io/bitnami/nginx:latest</code>	pas d'alias → cherche dans <code>unqualified-search-registries</code> ; un seul registry sur Ubuntu (<code>docker.io</code>), donc résolu ; plusieurs sur Fedora, donc question
<code>registry.masociete.be:5000/socle/nginx:1.25</code>	inchangé	nom complet : aucune résolution

La règle : si la première partie (avant le premier `/`) contient un **point** ou un **deux-points** (port), ou vaut `localhost`, c'est un registry. Sinon, c'est un namespace sur le registry par défaut.

Pourquoi. `masociete.be` ne peut pas être un nom d'utilisateur Docker Hub (les points y sont interdits), et un port n'a de sens que pour un serveur. Docker applique cette règle puis complète silencieusement ; Podman applique la même règle mais refuse de compléter à l'aveugle, parce que `nginx` sur `docker.io` et `nginx` sur `registry.interne` peuvent être deux images différentes.

Nuance. `bitnami/nginx` n'est **pas** une image officielle (namespace `bitnami`, pas `library`), malgré son nom. Et une image que vous construisez sans registry devient `localhost/...` : c'est un nom complet, avec `localhost` comme « registry » fictif.

Exemple.

```
podman pull nginx 2>&1 | head -2          # Resolved "nginx" as an alias ...
docker.io/library/nginx:latest
podman image inspect --format '{{.RepoTags}}' nginx
podman build -t api:1.0 . && podman images | grep api    # localhost/api 1.0
```

Question 2 — Même tag, contenu différent

Réponse. Le tag `2.3` a été **déplacé** entre-temps : quelqu'un a reconstruit et repoussé une image sous le même nom. A garde l'ancienne image (aucun pull), B a reçu la nouvelle. On le prouve en comparant les digests ; on l'évite en ne réutilisant jamais un tag publié et en déployant par digest.

Pourquoi. Un tag est un pointeur mutable côté registry. Le `pull` compare le digest distant au digest local et ne télécharge que s'ils diffèrent. Rien n'avertit qu'un tag a bougé.

Nuance. Le déplacement peut être involontaire : un pipeline qui pousse `api:2.3` à chaque exécution sur la branche de version, ou un `latest`. L'image de base peut aussi avoir bougé sans que votre Dockerfile change (`FROM eclipse-temurin:21-jre`) : la reconstruction « du même code » donne une image différente.

Exemple.

```
# sur A et sur B :
podman image inspect --format '{{.Digest}}' monapp/api:2.3
# digests différents -> le tag a bougé. Déploiement correct :
podman pull registry.interne/monapp/api@sha256:9d0d1f1e...
```

Question 3 — 62 Go affichés, disque intact

Réponse. Non. `SIZE` indique la taille **virtuelle** de chaque image, couches partagées comprises : les couches communes (JRE, Alpine, Debian) sont comptées dans chaque image mais stockées une seule fois. `podman system df` donne l'occupation réelle. Les fichiers sont dans `~/.local/share/containers/storage` — dans le disque virtuel de la distribution WSL (`ext4.vhdx`), pas dans un répertoire Windows.

Pourquoi. Le pilote `overlay` stocke chaque couche une fois, identifiée par son contenu, et les images ne sont que des listes de couches. Douze images Spring Boot sur le même JRE partagent ses 180 Mo.

Nuance. Le `vhdx` de WSL **grossit** automatiquement mais ne **rétrécit pas** tout seul quand vous supprimez des images : l'espace est libéré côté Linux, pas côté Windows, tant que vous n'avez pas compacté le disque (`wsl --shutdown` puis `Optimize-VHD` ou `diskpart`). C'est une surprise fréquente sur un poste Windows.

Exemple.

```
podman system df                # SIZE réel et RECLAIMABLE
podman system df -v | head      # colonne SHARED SIZE par image
podman info --format '{{.Store.GraphRoot}}' # /home/<vous>/.local/share/containers/storage
```

Question 4 — Le `rm` qui ne supprime rien

Réponse. Il a tort. `COPY` crée une couche qui **contient** `credentials.json`. Le `RUN ... rm` crée une couche ultérieure qui contient un *whiteout* masquant le fichier. L'image finale contient les deux couches : le fichier est présent, simplement invisible depuis un conteneur.

Pourquoi. Les couches sont immuables et additives. Une couche ne peut pas retirer un fichier d'une couche précédente ; elle ne peut que le masquer. Quiconque a l'image peut la sauvegarder avec `podman save`, extraire la couche du `COPY` et lire le fichier.

Nuance. Même dans un **seul** `RUN` (`COPY` puis `rm` dans la même instruction), le `COPY` reste une instruction séparée avec sa couche. Seuls un *build secret* (`RUN --mount=type=secret`) ou un multi-stage (labo 05) évitent la présence du fichier dans l'image finale.

Exemple.

```
podman save --format oci-archive -o img.tar mon-image:1.0
mkdir x && tar -xf img.tar -C x
for b in x/blobs/sha256/*; do tar -tf "$b" 2>/dev/null | grep -q credentials.json && echo
"présent dans $b"; done
```

Question 5 — 310 Mo, 61 Mo transférés

Réponse. Le registry possède déjà les couches inchangées (JRE, système, dépendances) ; le `push` ne transfère que les couches nouvelles — le JAR et ce qui suit — soit 61 Mo. Si tout était retransféré, c'est que la première couche de l'image change à chaque build : un `COPY . .` trop tôt, ou une instruction à contenu variable (date, version) avant les couches lourdes.

Pourquoi. Chaque couche a un digest. Avant d'envoyer un blob, le client demande au registry s'il l'a (`HEAD /v2/<repo>/blobs/<digest>`). Podman l'affiche moins explicitement que Docker (pas de `Layer already exists`), mais le transfert est instantané.

Nuance. Le partage entre *dépôts* du même registry dépend de son implémentation (Harbor et Docker Registry le font via *cross-repository mount*). Et une couche « inchangée » doit l'être bit à bit : un `RUN apt-get update` sans version épinglée produit une couche différente à chaque build.

Exemple.

```
podman push --tls-verify=false localhost:5000/socle/demo:1.1 # instantané : blobs déjà
présents
podman history mon-api:2.0 --format 'table {{.Size}}\t{{.CreatedBy}}' # repérer la couche qui
change
```

Question 6 — Deux tags, une image, et un fantôme

Réponse. Deux images distinctes : `f3a1b9c02d11` (portant les tags `2.0` et `1.9`) et `8b2c74e91a03` (sans nom). La ligne `<none>` est l'image *dangling* : un tag (`2.0` , probablement) a été reconstruit et déplacé, l'ancienne image a perdu son nom. `podman rmi api:1.9` retire **seulement le tag** (`Untagged: localhost/api:1.9`) : les données restent, référencées par `2.0` . `localhost/` est le registry fictif de toute image construite ou taguée sans nom de registry.

Pourquoi. Un `IMAGE ID` est le digest de la configuration de l'image ; deux lignes avec le même ID sont deux noms pour un contenu. Les données ne partent qu'avec le dernier nom.

Nuance. Ne confondez pas *dangling* (`<none>:<none>` , sans aucun tag) et *unused* (avec tag, mais sans conteneur). Un `podman image prune` sans `-a` ne supprime que les premières.

Exemple.

```
podman rmi api:1.9 # Untagged: localhost/api:1.9 (pas de Deleted)
podman images --filter dangling=true -q # 8b2c74e91a03
podman rmi 8b2c74e91a03 # Deleted: ... cette fois les données partent
```

Question 7 — `exec format error`

Réponse. L'image a été construite pour `linux/arm64` (Apple Silicon) et le serveur est `linux/amd64` : le noyau ne peut pas exécuter le binaire. Dépannage immédiat : reconstruire avec `--platform linux/amd64` (émulation QEMU, lent mais fonctionnel). Solution durable : faire

construire les images par la **CI** sur des agents `amd64`, ou produire des images multi-architecture (`podman manifest`).

Pourquoi. Un tag multi-architecture est une liste de manifestes ; au `build`, le moteur produit un manifeste pour l'architecture de la machine. Au `pull`, le serveur choisit l'entrée `amd64` ... qui n'existe pas, donc reçoit `arm64`.

Nuance. Les images *officielles* sont presque toutes multi-arch, ce qui masque le problème jusqu'au premier build maison. Et l'erreur peut survenir plus tôt : `podman run` sur le Mac d'une image `amd64` fonctionne (émulation), mais 5 à 10 fois plus lentement.

Exemple.

```
podman image inspect --format '{{.Architecture}}' registry.interne/api:1.4 # arm64
podman build --platform linux/amd64 -t registry.interne/api:1.4 .
```

Question 8 — `save` contre `export`, et le format OCI

Réponse. `save` exporte une **image** : couches, manifeste, configuration, tags. `export` exporte le système de fichiers d'un **conteneur**, aplati en une seule arborescence, sans métadonnées. Pour transporter une image Spring Boot vers un site isolé, `save` est le bon choix. Avec `export`, on perd `ENTRYPOINT`, `CMD`, `ENV`, `EXPOSE`, `USER`, `WORKDIR` — l'image importée ne sait plus comment démarrer — ainsi que les couches (plus de partage ni de cache). `--format oci-archive` produit la même image dans la disposition standard OCI (`blobs/sha256/`, `index.json`) au lieu du format historique de Docker.

Pourquoi. `export` ne voit que le résultat de l'assemblage des couches, comme un `tar` fait depuis l'intérieur du conteneur. La configuration vit dans l'image, pas dans le système de fichiers.

Nuance. `export` a un usage légitime : récupérer un système de fichiers pour analyse, ou fabriquer une image « à plat » à partir d'un conteneur configuré à la main (mauvaise pratique, mais documentée). Le format `docker-archive` reste le plus courant ; `oci-archive` est celui à utiliser si le destinataire n'est pas Docker (Kubernetes via `ctr`, skopeo...).

Exemple.

```
podman save --format oci-archive -o api.tar mon-api:1.0
podman load -i api.tar # Loaded image: localhost/mon-api:1.0
podman export c1 | podman import - plat:1 # Config.Cmd = null
```

Question 9 — Le second `pull`

Réponse. Le moteur a demandé au registry le **manifeste** du tag, comparé son digest à celui de l'image locale, constaté qu'ils sont identiques, et n'a rien téléchargé d'autre. Le manifeste fait quelques kilo-octets : le coût réseau est celui d'une ou deux requêtes HTTP.

Pourquoi. Chaque couche et le manifeste sont adressés par contenu ; le client sait exactement ce qu'il possède. Un pull est toujours différentiel, et à la limite, vide.

Nuance. Podman ne dit pas « Image is up to date » : il affiche simplement l'identifiant de l'image. Et « moins d'une seconde » suppose un registry proche ; face à Docker Hub, la requête peut prendre plusieurs secondes de latence, sans rien télécharger pour autant. Enfin, cette requête compte dans le quota de Docker Hub (question 10).

Exemple.

```
time podman pull alpine          # d529dd0c... - quelques secondes de réseau, zéro couche transférée
```

Question 10 — toomanyrequests

Réponse. Docker Hub plafonne les `pull` anonymes par adresse IP (et par compte pour les utilisateurs authentifiés). Les agents de CI sortent tous par la même IP publique : le quota est partagé par toute l'entreprise, d'où des échecs « aléatoires » selon la charge du moment. Les deux réponses : un **pull-through cache** (registry interne qui met en cache Docker Hub) et/ou la **copie interne** des images de base dans le registry d'entreprise (`skopeo copy`), avec des Dockerfiles qui référencent ce registry.

Pourquoi. Chaque `pull` interroge au moins le manifeste, même quand l'image est déjà locale. Une CI qui reconstruit cent fois par jour dépasse vite le quota.

Nuance. S'authentifier (`podman login docker.io`) relève le quota mais ne l'annule pas, et met un compte personnel dans la CI. La copie interne a un avantage supplémentaire : on contrôle *ce qui entre* (scan, validation), et on ne dépend plus de la disponibilité de Docker Hub.

Exemple.

```
skopeo copy docker://docker.io/library/node:22-alpine docker://registry.interne/socle/node:22-alpine
# puis dans le Dockerfile : FROM registry.interne/socle/node:22-alpine
```

Question 11 — HTTP contre HTTPS

Réponse. Docker traite `localhost` (et `127.0.0.0/8`) comme un registry *insecure* par défaut : il accepte le HTTP sans rien dire. Podman n'a pas d'exception : tout registry doit présenter un certificat TLS valide. Deux façons de passer : `--tls-verify=false` sur la commande, ou une entrée `[[registry]] location = "localhost:5000" insecure = true` dans `registries.conf`. La seconde ne doit jamais atteindre un fichier versionné ou déployé sur un serveur : elle désactive la vérification pour **tous** les usages de ce registry, silencieusement.

Pourquoi. Un registry sans TLS peut être usurpé par n'importe qui sur le chemin réseau (*man in the middle*) et renvoyer une image piégée. Sur `localhost`, le risque est faible ; Podman préfère que vous le disiez plutôt que de le supposer.

Nuance. `--tls-verify=false` désactive aussi la vérification du **certificat** sur un registry HTTPS auto-signé — la bonne pratique est plutôt d'installer le certificat de l'autorité interne dans `/etc/containers/certs.d/<registry>/ca.crt`.

Exemple.

```
podman push --tls-verify=false localhost:5000/socle/demo:1.0      # explicite, visible dans l'historique
# OU, pour un poste de développement seulement :
printf '[[registry]]\nlocation = "localhost:5000"\ninsecure = true\n' >>
~/.config/containers/registries.conf
```

Question 12 — image is in use by a container

Réponse. Un conteneur (créé à partir de cette image, même arrêté) existe encore ; l'image est sa couche de base. Le moteur refuse parce que supprimer l'image casserait ce conteneur. Proper : lister ce conteneur (`podman ps -a --filter ancestor=...`), le supprimer si son état ne sert plus, puis `rmi`. `podman rmi -f` supprime le tag et l'image **et** le conteneur qui en dépend, sans demander : on perd les logs, la couche d'écriture et la possibilité d'inspecter le conteneur — pour gagner dix secondes.

Pourquoi. Un conteneur est image + couche d'écriture. Sans l'image, la couche d'écriture ne veut plus rien dire.

Nuance. Le message de Podman parle de « conteneurs externes » : ceux créés par Buildah ou par un autre outil partageant le même stockage, invisibles dans `podman ps`. `podman ps -a --external` les montre. Docker, lui, refuse aussi mais son message cite l'identifiant du conteneur en clair.

Exemple.

```
podman ps -a --filter ancestor=mon-api:1.0 --format '{{.Names}} {{.Status}}'
podman logs <conteneur> > incident.log      # on sauve ce qui doit l'être
podman rm <conteneur> && podman rmi mon-api:1.0
```

Question 13 — Les couches à 0B

Réponse. Ce sont des instructions de **métadonnées** — `ENV`, `CMD`, `ENTRYPOINT`, `EXPOSE`, `LABEL`, `USER`, `WORKDIR` — qui modifient la configuration de l'image sans toucher au système de fichiers. Elles apparaissent dans l'historique parce que chaque instruction laisse une trace, même vide. La couche de 180 Mo (un `RUN apt-get install`, un `COPY` de JRE) est la seule qui compte : `podman history` désigne la ligne à optimiser.

Pourquoi. Une image est un tableau d'instructions avec, pour certaines, un blob de fichiers associé. Le poids vient uniquement des blobs.

Nuance. Une couche non vide peut cacher une suppression : `RUN rm -rf /var/lib/apt/lists/*` dans un `RUN` séparé pèse presque 0B mais ne récupère rien. `history` montre le coût de chaque étape ; `podman image tree` montre en plus quelles couches viennent de l'image de base.

Exemple.

```
podman history nginx:alpine --format 'table {{.Size}}\t{{.CreatedBy}}'    # 12 lignes à 0B, une à 50.7MB
podman image tree nginx:alpine
```

Question 14 — Trois stratégies de tags

Réponse. (a) `latest` réécrit : **retour arrière impossible** (l'ancienne image n'a plus de nom) et diagnostic impossible (impossible de savoir quelle version tournait). (b) `1.4.2` : retour arrière vers `1.4.1` immédiat ; diagnostic correct si le tag est immuable, mais deux builds de `1.4.2` (correctif rapide) peuvent coexister sans qu'on les distingue. (c) `1.4.2-b318-a9f3c21` : chaque image est unique, on remonte au commit exact et au build ; retour arrière vers n'importe quel build antérieur. Le coût est un nom illisible et une rétention à gérer.

Pourquoi. Le déploiement et le diagnostic ont besoin d'une correspondance **bijective** entre un nom et un contenu. Seul (c) la garantit par construction ; (b) la garantit par discipline ; (a) l'interdit.

Nuance. Les trois coexistent souvent : la CI pousse (c) ; un tag (b) est *ajouté* sur la même image quand elle est validée ; `latest` n'existe que pour le confort des développeurs, jamais dans un manifeste de déploiement. Et le déploiement lui-même épingle le digest.

Exemple.

```
podman tag registry.interne/api:1.4.2-b318-a9f3c21 registry.interne/api:1.4.2    # même image,
second nom
podman image inspect --format '{{.Digest}}' registry.interne/api:1.4.2        # ce qu'on
déploie réellement
```